

# 第二次作业

## 一、作业总体内容

具体内容如下 2-1, 2-2, 2-3:

## 二、各分项作业内容

### (一) 作业 2-1: makefile 工程管理器的使用

**内容:** 编写一个包含多文件的 Makefile

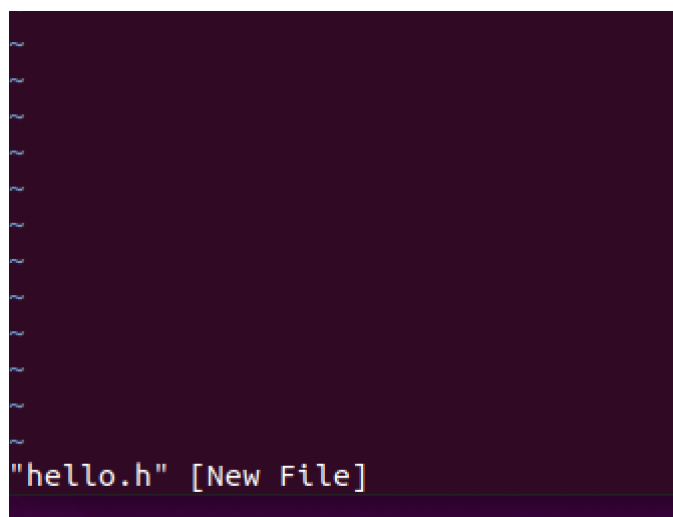
**目的:** 通过对包含多文件的 Makefile 的编写, 熟悉各种形式的 Makefile, 并且进一步加深对 Makefile 中用户自定义变量、自动变量及预定义变量的理解

**平台:** PC 机, Ubuntu 操作系统, gcc 等工具

**具体步骤:**

用 vi 在同一目录下编辑两个简单的 hello 程序

```
ulmus@Ulmus:~$ mkdir -p ~/embedded/2-1 && cd ~/embedded/2-1
```



Hello.h

```
ulmus@Ulmus: ~/embedded/2-1
#include <stdio.h>
void print_hello();
~
~
~
~
~
~
~
```

Hello.c

```
ulmus@Ulmus:~/embedded/2-1$ vi hello.c

#include "hello.h"
int main(){
    print_hello();
    return 0;
}
void print_hello(){
    printf("Hello everyone!\n");
}
~
~
```

仍在同一目录下用 vi 编辑 Makefile，不使用变量替换，用一个目标体实现（即直接将 hello.c 和 hello.h 编译成 hello 目标体），并用 make 验证所编写的 Makefile 是否正确

```
ulmus@Ulmus:~/embedded/2-1$ vi Makefile
```

不使用变量替换

```
hello: hello.c hello.h
    gcc hello.c -o hello
clean:
    rm -f hello
~
~
```

Make 指令

成功

```
ulmus@Ulmus:~/embedded/2-1$ make  
gcc hello.c -o hello
```

```
ulmus@Ulmus:~/embedded/2-1$ ./hello  
Hello everyone!
```

```
ulmus@Ulmus:~/embedded/2-1$ make clean  
rm -f hello
```

将上述 Makefile 使用变量替换实现，同样用 make 验证所编写的 Makefile 是否正确

```
TARGET = hello  
SRCS = hello.c  
HDRS = hello.h  
CC = gcc  
$(TARGET): $(SRCS) $(HDRS)  
            $(CC) $(SRCS) -O $@  
clean:  
        rm -f $(TARGET)  
~
```

```
gcc      hello.c      -o hello  
ulmus@Ulmus:~/embedded/2-1$
```

```
ulmus@Ulmus:~/embedded/2-1$ ./hello  
Hello everyone!  
ulmus@Ulmus:~/embedded/2-1$ make clean  
rm -f hello  
ulmus@Ulmus:~/embedded/2-1$
```

编辑另一 Makefile，取名为 Makefile1，不使用变量替换，但用两个目标体实现（也就是首先将 hello.c 和 hello.h 编译为 hello.o，再将 hello.o 编译为 hello），再用 make 的“-f”选项验证这个 Makefile1 的正确性

将上述 Makefile1 使用变量替换实现

相关文件参考代码：

```
#hello.c
```

```
print("Hello everyone!\n")
```

```
#hello.h
```

```
#include <stdio.h>
```

```
hello.o: hello.c hello.h^M
    gcc -c hello.c -o hello.o  ^M
hello: hello.o^M
    gcc hello.o -o hello^M
clean:^M
    rm -f hello hello.o
~
```

```
ulmus@Ulmus:~/embedded/2-1$ make -f Makefile1
gcc -c hello.c -o hello.o
```

```
CC = gcc^M
TARGET = hello^M
OBJ = hello.o    # 新增变量：目标文件^M
SRCS = hello.c^M
HDRS = hello.h^M
^M
$(TARGET): $(OBJ)^M
    $(CC) $(OBJ) -o $(TARGET)^M
$(OBJ): $(SRCS) $(HDRS)^M
    $(CC) -c $(SRCS) -o $(OBJ)^M
clean:^M
    rm -f $(TARGET) $(OBJ)
~
```

```
ulmus@Ulmus:~/embedded/2-1$ vi Makefile1
ulmus@Ulmus:~/embedded/2-1$ make -f Makefile1
gcc -c hello.c -o hello.o
gcc hello.o      -o hello
```

## （二）作业 2-2: GDB 调试工具的使用

内容：将原来出错的程序经 GDB 调试，找出 bug，修改源码，输出正确的倒序显示字符串的结果

目的：通过调试一个有问题的程序，进一步熟练用 GDB 操作，熟练使用 gcc 编译命令及 gdb 的调试命令，通过对有问题程序的跟踪调试，进一步提高发现问题和解决问题的能力

平台：带有 Linux 操作系统的 PC 机

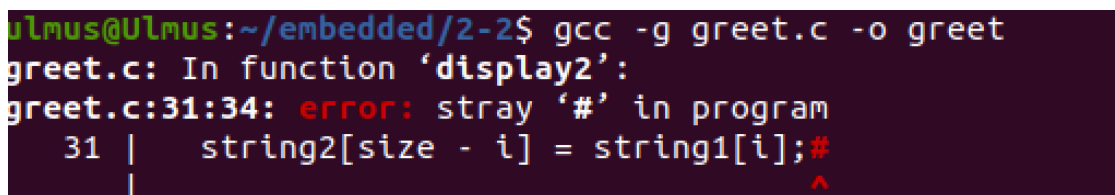
步骤：

使用 vi 编辑器，生成 greet.c 的文件，此代码的原意为输出倒序输出 main 函数中定义的字符串，但结果显示没有输出

使用 gcc 编译这段代码

运行生成的可执行文件 greet，观察运行结果

实验结果：gcc 编译失败



```
ulmus@Ulmus:~/embedded/2-2$ gcc -g greet.c -o greet
greet.c: In function 'display2':
greet.c:31:34: error: stray '#' in program
 31 | string2[size - i] = string1[i];#
    |                                ^
```

使用 gdb 调试程序，通过设置断点、单步跟踪，一步步找出错误所在

纠正错误，更改源程序并得到正确的结果

参考调试步骤：

启动 gdb 调试：gdb greet

查看源代码

在 for 循环处设置断点

在 printf 函数处设置断点

查看断点设置情况

运行代码

单步运行代码

查看暂停点变量值

继续单步运行代码数次，并使用命令查看出错语句

退出 gdb

重新编辑 greet.c

重新编译 greet.c

```
ulmus@Ulmus:~/embedded/2-1$ mkdir -p ~/embedded/2-2
ulmus@Ulmus:~/embedded/2-1$ cd ~/embedded/2-2
ulmus@Ulmus:~/embedded/2-2$ vi greet.c
```

```
ulmus@Ulmus:~/embedded/2-2$ gcc -g greet.c -o greet
greet.c: In function 'display2':
greet.c:31:34: error: stray '#' in program
  31 |     string2[size - i] = string1[i];#
      |                                   ^
```

```
ulmus@Ulmus:~/embedded/2-2$ gdb greet
```

无法调试，先对影响编译的问题进行修正。

```
ulmus@Ulmus:~/embedded/2-2$ gcc -g greet.c -o greet
greet.c: In function 'display2':
greet.c:31:34: error: stray '#' in program
  31 |     string2[size - i] = string1[i];#
      |                                   ^
```

显示多了#号，进行修复，生成了可执行文件

```
ulmus@Ulmus:~/embedded/2-2$ gcc -g greet.c -o greet
```

```
ulmus@Ulmus:~/embedded/2-2$ ./greet
The original string is Embedded Linux
The string afterward is
```

进行调试

在三处打断点

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./greet...
(gdb) break main
Breakpoint 1 at 0x11c9: file greet.c, line 9.
(gdb) break display1
Breakpoint 2 at 0x123d: file greet.c, line 19.
(gdb) break display2
Breakpoint 3 at 0x1268: file greet.c, line 24.
```

n

```
Starting program: /home/ulmus/embedded/2-2/greet
Breakpoint 1, main (argc=21845, argv=0x7ffff7fb52e8 <__exit_funcs_lock>)
  at greet.c:9
 9      {
(gdb) n
10          char string[] = "Embedded Linux";
(gdb)
```

出现问题

```
(gdb) next
display1 (string=0x7ffffffffffde79 "Embedded Linux") at greet.c:21
21      }
(gdb) next
main (argc=1, argv=0x7ffffffffffdf88) at greet.c:13
13      display2 (string);
(gdb) step

Breakpoint 3, display2 (
  string1=0x555555555265 <display1+40> "\220\311\303\363\017\036\372UH\211\345
H\203\354 H\211}\350H\213E\350H\211\307\350\034\376\377\377\211E\364\213E\364\20
3\300\001H\230H\211\307\350\071\376\377\377H\211E\370\307", <incomplete sequence
\360>) at greet.c:24
24      {
(gdb) step
28          size = strlen (string1);
(gdb) step
The string afterward is
[Inferior 1 (process 4760) exited normally]
(gdb) step
The program is not being run.
(gdb) step
The program is not being run.
(gdb)
```

display2 函数中:

反转字符串后未添加'\0'（字符串结束符），导致 printf 无法正确识别结尾。

错误地给 string2[size+1]赋值（越界，string2 大小为 size+1，最大索引为 size）。

正确做法：将 string2[size+1] = ' ' 改为 string2[size] = '\0'（添加结束符）。

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <stdlib.h>
4.
5. int display1(char *string);
6. int display2(char *string);
7.
8. int main(int argc, char **argv) {
9.     char string[] = "Embedded Linux";
10.    display1(string);
11.    display2(string);
12.    return 0;
13. }
14.
15. int display1(char *string) {
16.    printf("The original string is %s\n", string);
17.    return 0;
18. }
19.
20. int display2(char *string) {
21.    char *string2;
22.    int size, i;
23.    size = strlen(string);
24.    string2 = (char *)malloc(size + 1);
25.
26.    // 修正 1: 正确倒序索引 (size-1-i)
27.    for (i = 0; i < size; i++) {
28.        string2[size - 1 - i] = string[i];
29.    }
30.    // 修正 2: 正确设置结束符 (\0+下标 size)
31.    string2[size] = '\0';
32.
33.    printf("The string afterward is %s\n", string2);
34.    free(string2);
35.    return 0;
36. }
```



37.

38.

修改后运行如下

```
ulmus@Ulmus:~/embedded/2-2$ gcc -g greet.c -o greet
ulmus@Ulmus:~/embedded/2-2$ ./greet
The original string is Embedded Linux
The string afterward is xuniL deddebME
ulmus@Ulmus:~/embedded/2-2$
```

### (三) 作业 2-3: Linux 系统 UDP 网络协议编程

目的: 熟悉 socket 网络编程的基本方法

平台: Vmware 虚拟机, Ubuntu 操作系统, gcc 编译器

内容: 通过一个简单的 UDP 服务器端, 接收客户端的连接请求, 并接受客户端发来的信息

具体步骤:

建立目录: `mkdir/workdir/linux/application/udp`

将 `client.c` 和 `server.c` 拷贝到上述文件夹下

```
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ sudo cp /media/sf_share/client.c .
[sudo] password for ulmus:
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$

ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ sudo cp /media/sf_share/server.c .
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$
```

分别编译 c 文件, 生成可执行文件 `client` 和 `server`

执行文件: `./server 10.0.2.15 8888` (各自 ubuntu 系统的 IP 可通过 `ifconfig` 查询)

按下 `ctrl+Alt+t`, 打开另一个终端, 进入该目录, 运行客户端程序: `./client 10.0.2.15 8888`

在客户端提示符下输入任意字符串, 如: `hello`, 查看服务器终端是

否能接收该信息并显示

```
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ sudo gcc server.c -o server
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ sudo gcc client.c -o client
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
  inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft 0
  inet6 ::1/128 scope host
    valid_lft forever preferred_lft 0
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 link/ether 08:00:27:02:a8:de brd ff:ff:ff:ff:ff:ff
  inet 10.0.2.15/24 brd 10.0.2.255 scope global enp0s3
    valid_lft 84102sec preferred_lft 0
  inet6 fd00::cf0f:7d3b:32a9:5e6d/64 scope global dynamic mcastflags
    valid_lft 86203sec preferred_lft 0
  inet6 fd00::ec76:2b22:3f97:7605/64 scope global dynamic mcastflags
    valid_lft 86203sec preferred_lft 0
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ cd /mkdi/workdir/linux/application/udp
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ ./client 10.0.2.15 8888
bash: ./client: No such file or directory
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ ./client 10.0.2.15 8888
```

在客户端输入 hello 等多字符串

```
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ ./client 10.0.2.15 8888
bash: ./client: No such file or directory
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$ ./client 10.0.2.15 8888
hello
hello
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$
from 10.0.2.15:53643 hello
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$
from 10.0.2.15:53643 ooad
ulmus@Ulmus:/mkdi/workdir/linux/application/udp$
from 10.0.2.15:53643 ooad
```

作业要求:

根据具体内容，完成客户端向服务器发送信息的要求；可尝试发送不同的字符串信息

剖析代码 client.c 和 server.c，加入注释

#### （四）作业 2-4：Linux 系统多进程实验

前期准备：

将【华清远见-嵌入式 ARM 实验箱资料\程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录 D:\share 下

将 09.Linux 系统 fork 等系统调用实验实验部分代码拷贝到虚拟机 Linux 下

具体步骤：

建立相关目录：LinuxBubuntu64-vm:~\$ mkdir -p 相关目录路径

将代码从共享目录拷入虚拟机 Linux 操作系统下

```
ulmus@Ulmus:~$ sudo cp /media/sf_share/fork_test/93.c ~/embedded/2-4
[sudo] password for ulmus:
ulmus@Ulmus:~$
ulmus@Ulmus:~/embedded/2-4$ gcc 93.c -o 93
ulmus@Ulmus:~/embedded/2-4$
```

执行代码，观察结果：\$ ./ 代码文件名 执行观察结果

```
ulmus@Ulmus:~/embedded/2-4$ ./93
the test content!
fork test!
global=24 test=2 Parent,my PID is 4574
global=23 test=1 Child,my PID is 4575
ulmus@Ulmus:~/embedded/2-4$
```

实验核心内容：

fork 后父子进程会同步运行，但父子进程的返回顺序是不确定的。

设两个变量 global 和 test 来检测父子进程共享资源的情况

通过观察运行结果，跟踪 global 和 test 变量，体会 Linux 多进程调度的一些机制

相关文件参考代码片段：

```
int global=0, test=0;

pid_t fork ();

if (pid==0)

{

global++;test++;

printf ("global=%d test=%d Child, my PID is %d\n", global, test, getpid ());

}

else

{

global+=2;test+=2;

printf ("global=%d test=%d Parent, my PID is %d\n", global, test, getpid ());

}
```

（注：参考代码以 93.c 为核心实验文件）

### 三、作业提交要求

命名格式：作业命名为“姓名+学号+2.zip”（任意压缩软件）

包含文件：

pdf 文档 1 份，命名：作业 2.pdf，包含：作业 2-1 中的实操过程截屏说明部分+作业 2-2 中的说明文档部分（阐述清楚查找 bug 的调试过程并附必要截屏）+作业 2-3 中操作过程截屏部分+作业 2-4 中实

操过程截屏+实验结果分析和体会

Makefile 文件、Makefile1 文件

greet.c 源码 1 份

注释后的 client.c、server.c 和 93.c，分别命名为：

client\_mark.c、server\_mark.c、93\_mark.c

提交时间：2025.10.19 晚 12 点

提交路径：course.xmu.edu.cn